ECE 385

Spring 2025

Lab 9

VGA Text Mode Controller with Avalon-MM Interface

Cristina Tobing 3220300398
Artha Amanda 3220300399
TA: Qiyang Wu

Contents

ECE 385 1

VGA Text Mode Controller with Avalon-MM Interface 1

1. Introduction 3

a. Operation of the VGA Interface and Our Design Goal 3

b. Improvement Based on Lab 8 3

2. Description of Lab 9 System 3

a. Week 1: Monochrome Text Display 3

b. Week 2: Color Text Display 5

3. Block diagram 7

4. Module Description 9

5. Design Resources and Statistics 12

Week 1 12

Week 2 12

6. Conclusion 13

Functionality Discussion 13

Potential Extensions 13

Lab Difficulty and Improvements 13

1. Introduction

a. Operation of the VGA Interface and Our Design Goal

For this lab, we built a basic graphics controller that works in text mode and hooks up to the Avalon memory-mapped bus. It outputs text to a VGA display in an 80-column format. The project was done over two weeks. During the first week, we made a simple version that only shows black and white characters, kind of like an old-school monochrome display. Then in the second week, we upgraded the controller so it could show characters in different colors, since we had access to more video memory.

b. Improvement Based on Lab 8

We started off by modifying the system from Lab 8. We took out parts we didn't need, like the USB interface, the interval timer, and the PIOs. For week one, we used basic registers and switched over to on-chip memory later on to handle communication between our software and hardware parts. We also made a custom IP block called vga_test_ctrl, built the logic for it, and set up the interface using the component editor.

2. Description of Lab 9 System

a. Week 1: Monochrome Text Display

i. Written Description of the Entire Lab 9 System

In Lab 9, we designed and implemented a simplified text mode VGA controller with a focus on improving VRAM functionality across two stages (Lab 9.1 and Lab 9.2). Initially, in Lab 9.1, the VRAM was implemented using registers, which resulted in extremely long compilation times due to the limitations of routing and the number of registers available. This setup allowed the display of text, but the text could only be shown with a single foreground color and a single background color for each frame. The functionality of the system was limited by these constraints, although the displayed text could be unlimited.
In Lab 9.2, the VRAM was upgraded to on-chip memory, significantly expanding the available space. This change allowed us to enhance the text display by supporting a 16-color palette, enabling both the foreground and background colors to be chosen from this palette and dynamically updated through C code. Additionally, the text display could now be controlled more flexibly using C programming.

ii. High-Level Description of VGA Text Mode Controller IP

The VGA Text Mode Controller IP translates character data from VRAM into VGA display signals. It fetches glyph data from the font_rom.sv module, processes it according to the screen's raster scan, and handles color inversion using control bits. The controller is synchronized with the processor clock, but the pixel clock runs at 50% of the processor clock speed. It generates the horizontal and vertical sync signals, the blanking interval indicator, and the DrawX and DrawY coordinates for pixel positioning, using RGB values provided by the processor.

iii. Logic Used to Read and Write VGA Registers

The VGA controller interfaces with the Avalon-MM bus using signals like read, write, readdata, writedata, and byteenable to handle memory operations for 32-bit wide registers. Since the VGA wiring handles 8-bit data, 32-bit data is split into four 8-bit chunks, using data latches for each read/write operation. The system ensures a read latency of one cycle and a write latency of zero cycles, with byte-level control provided by the byteenable signal for precise data transactions.

iv. The algorithm used to Draw Text Characters from VRAM and font ROM

Characters are drawn by calculating VRAM addresses based on the current screen position (DrawX, DrawY). Each word in VRAM can store four characters, indexed in raster order. The character data consists of a 7-bit index for the font ROM and a 1-bit color inversion flag. The font ROM is accessed using the index to retrieve the corresponding 8x16 pixel bitmap.
The address of each character is determined by the formula (Col + Row * 80) / 4, which calculates the register storing the current character. The first 2 bits of the character identify the specific bit within the register. The global foreground and background color values are stored in separate registers, and the pixel data is processed to display the character at the correct position with the appropriate colors.

v. Implementation of the Inverse Color Bit and the Control Register

Each character stored in VRAM has one bit used to enable inverted color mode, which swaps the foreground and background colors during display. The global colors for foreground and background are set using a control register, where each RGB value takes 4 bits (total 12 bits per color). The register format packs these RGB values into specific bit ranges. When drawing, the system checks the font ROM output and the invert bit: if both match (both 0 or both 1), the background color is shown; otherwise, it switches to the foreground color. This lets individual characters appear with inverted colors while still using the global color settings.

b. Week 2: Color Text Display

i. Modification of register-based VRAM to on-chip memory-based VRAM. How did your design share the limited on-chip memory ports?

Instead of using individual registers for VRAM, which consumed excessive resources, the design was upgraded to use dual-port on-chip memory. This memory allows one port to handle reads and the other to handle writes simultaneously, improving efficiency. Avalon-MM signals like avl_read, avl_write, and byteenable are used to control access. By enabling concurrent memory operations, this approach makes better use of the FPGA's limited memory ports and supports smooth interaction between the VGA controller and the processor.

ii. Corresponding modifications to the Platform Designer IP (e.g. Part Editor)

The Avalon address width (AVL_ADDR) was increased from 10 to 12 bits to support the expanded VRAM size. Previously, each 32-bit word stored four characters, but the design was updated to store only two characters per word. This change doubled the memory requirement, prompting the need for a wider address space in the IP configuration within Platform Designer.

iii. Modified sprite drawing algorithm with the updated indexing equations from on-screen pixels to VRAM.

In Week 2, the design was updated by expanding the VRAM to 1200 entries and enabling per-character foreground and background color customization. Despite these upgrades, the core address computation remained similar to Week 1. The equation num = DrawY[9:4] * 80 + DrawX[9:3] still determines the character's position in memory. Instead of fetching character data from registers, it now comes from the RAM module. Using char, the system identifies which glyph to render and checks whether to invert the colors. The pixel data is then accessed via sprite_addr = {CODEn, DrawY[3:0]} and evaluated using (sprite_data[3'b111 - DrawX[2:0]] ^ IVn).

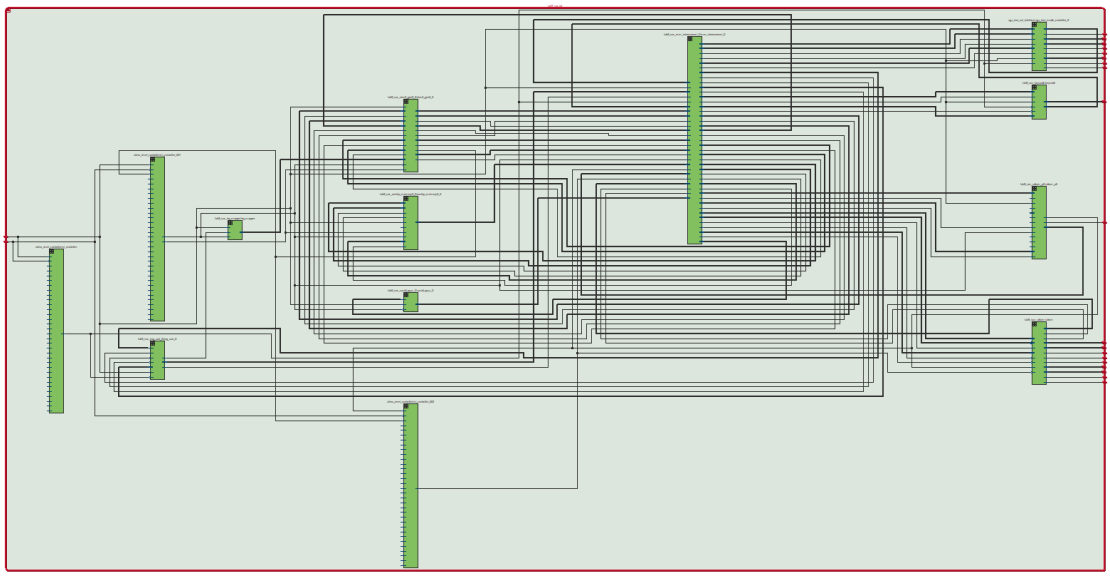
iv. Additional modifications necessary to support multicolored text

To enable multiple colors for characters, the VRAM structure was enhanced to hold color metadata along with the glyph index. This required increasing the Avalon-MM address width from 10 to 12 bits to accommodate the larger data format. A color palette system was introduced, using FPGA registers to store the RGB values of 16 selectable colors. Eight 32-bit registers, located at addresses 0x800 through 0x807, were reserved for this palette, with each register encoding two 12-bit RGB color entries. These palette entries allow dynamic assignment of foreground and background colors per character.

v. Additional hardware/code to draw palette colors

The color palette was implemented through eight 32-bit FPGA registers, occupying address space from 0x800 to 0x807. Each register encodes two 12-bit RGB values, representing a pair of selectable colors. When rendering text, each character references its color indices from VRAM, and the VGA controller uses those indices to retrieve RGB values from the palette. If the index is even, the lower 16 bits (right-side) of the register are selected; if odd, the upper 16 bits (left-side) are used, enabling efficient access to multiple colors for text rendering.

3. Block diagram



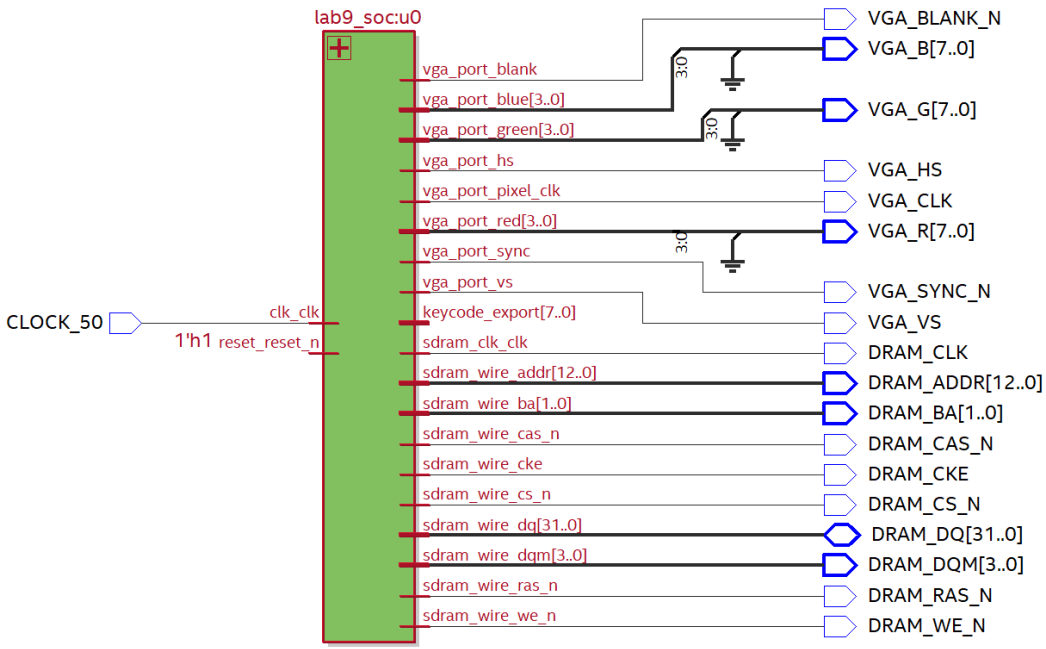
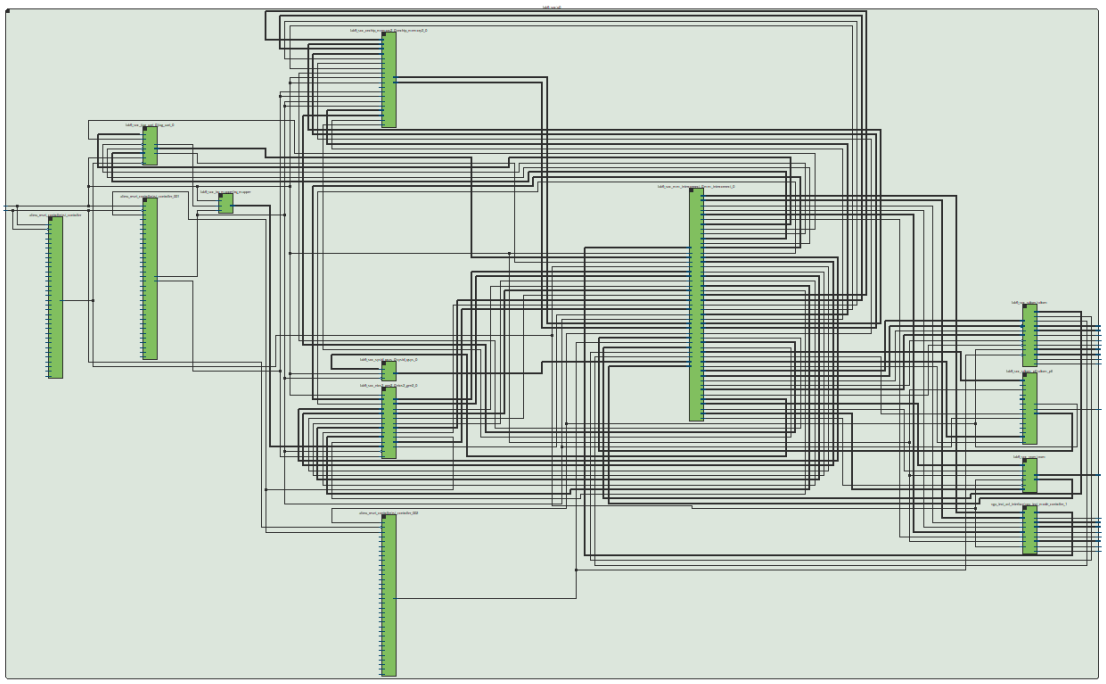


Figure 1: RTL diagram of VGA controller of 9.1



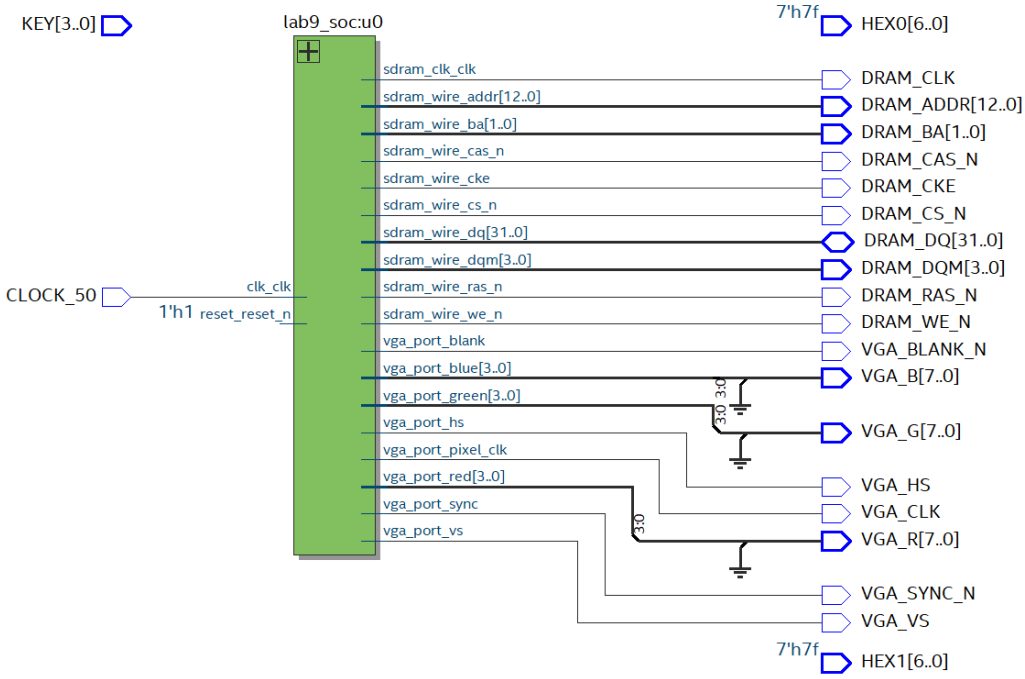


Figure 2: RTL diagram of VGA controller of 9.2

4. Module Description

Module: lab9.sv
Inputs: CLOCK_50, [3:0] KEY, [12:0] DRAM_ADDR, [31:0] DRAM_DQ, [1:0] DRAM_BA, [3:0] DRAM_DQM, DRAM_RAS_N, DRAM_CAS_N, DRAM_CKE, DRAM_WE_N, DRAM_CS_N, DRAM_CLK, [7:0] VGA_R, [7:0] VGA_G, [7:0] VGA_B, VGA_CLK, VGA_SYNC_N, VGA_BLANK_N, VGA_VS, VGA_HS, [6:0] HEX0, [6:0] HEX1,
Outputs: -
Description: The lab9 module acts as the main module for Lab 9, where everything comes together. It connects the VGA display and the SDRAM so they can work with the Nios II processor on the DE2-115 board. Its job is to make sure all the different parts of the system, like the memory and display, can talk to each other properly. It sets up the necessary hardware configurations and handles the signals needed to get the VGA output and memory access working smoothly on the FPGA.
Purpose: This module brings together all the key parts of the Lab 9 setup. It relies on the system clock to keep things like memory access and video output in sync. Inside, it sets up the VGA output signals and gets the SDRAM ready to read and write data, which is important for running programs on the Nios II processor. There's also a reset feature controlled by KEY[0], which lets you reset the system at any time in case something goes wrong during operation.

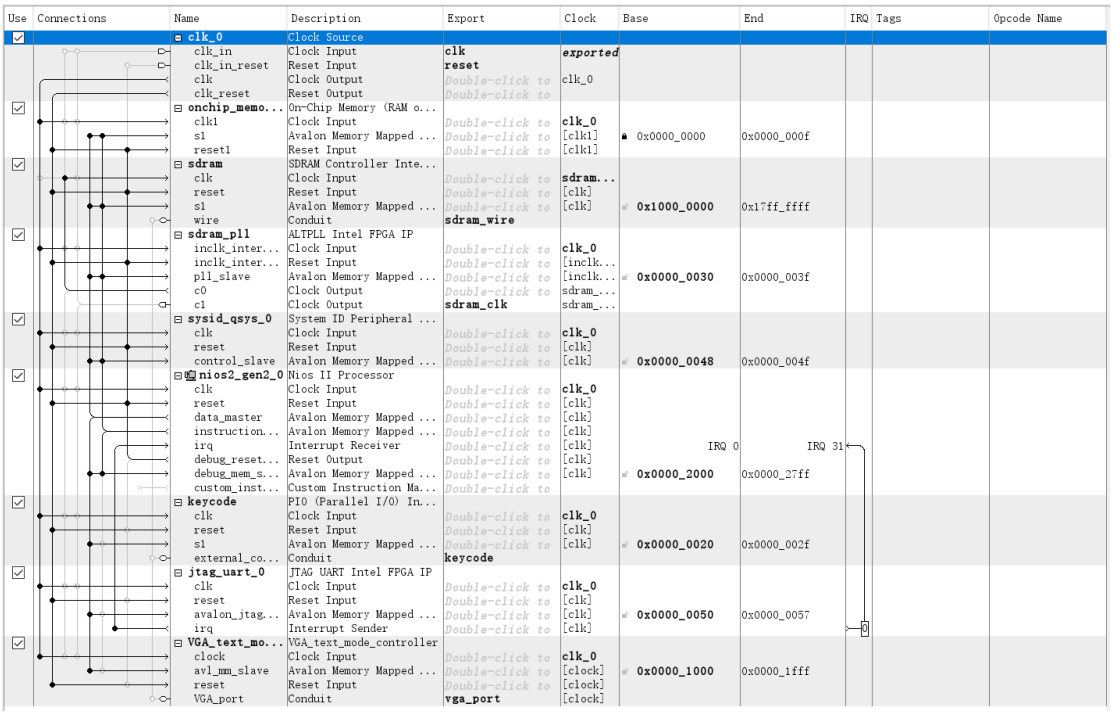
Module: dualport.v
Inputs: [11:0] address_a, [11:0] address_b, [3:0] byteena_a, clock, [31:0] data_a, [31:0] data_b, rden_a, rden_b, wren_a, wren_b,
Outputs: [31:0] q_a, [31:0] q_b
Description: The dualport module sets up a two-way memory system using the altsyncram block, tailored for the Cyclone IV E FPGA. It allows both ports—A and B—to read from or write to memory at the same time without interfering with each other. Each port can access up to 4096 entries, with each word being 32 bits wide. The setup uses a bidirectional dual-port mode, which makes it flexible for handling data from multiple sources or tasks at once.
Purpose: This module lets you write individual bytes using the byteena_a signal on port A, giving more control over what parts of a word get updated. It’s designed with parameters that can skip clocking on data inputs and outputs, helping reduce delays and improve speed. The memory starts out empty when powered on, which is useful for systems that need to manage memory contents manually from the beginning. Its settings also include special read and write modes that let it handle new data without blocking reads, making it a good fit for fast-paced applications where efficiency matters.

Module:VGA_controller.sv
Inputs: Clk, Reset
Outputs: hs, vs, pixel_clk, blank, sync, [9:0] DrawX, [9:0] DrawY,
Description: The vga_controller module handles all the timing signals needed to work with a 640x480 VGA screen. Instead of using the standard 25.175 MHz clock, it runs on a 25 MHz pixel clock, which slightly lowers the refresh rate. It takes care of generating the horizontal and vertical sync signals, along with a blanking signal that marks when the screen should or shouldn’t show

pixels. The module also outputs the pixel clock and the x and y coordinates that tell the display where to draw.
Purpose: This module keeps track of the current pixel position by counting across the screen and down each line, then starting over when it hits the edge of the display area. It creates a 25 MHz pixel clock by splitting the incoming 50 MHz clock in half. The horizontal and vertical sync signals follow VGA timing rules, but they're tweaked a bit to work with the slightly slower clock speed. A blanking signal is used to hide the parts of the screen that aren't supposed to be visible, which helps make the display look cleaner. The sync output isn't used in this project, but it's still in the design in case it's needed later on.

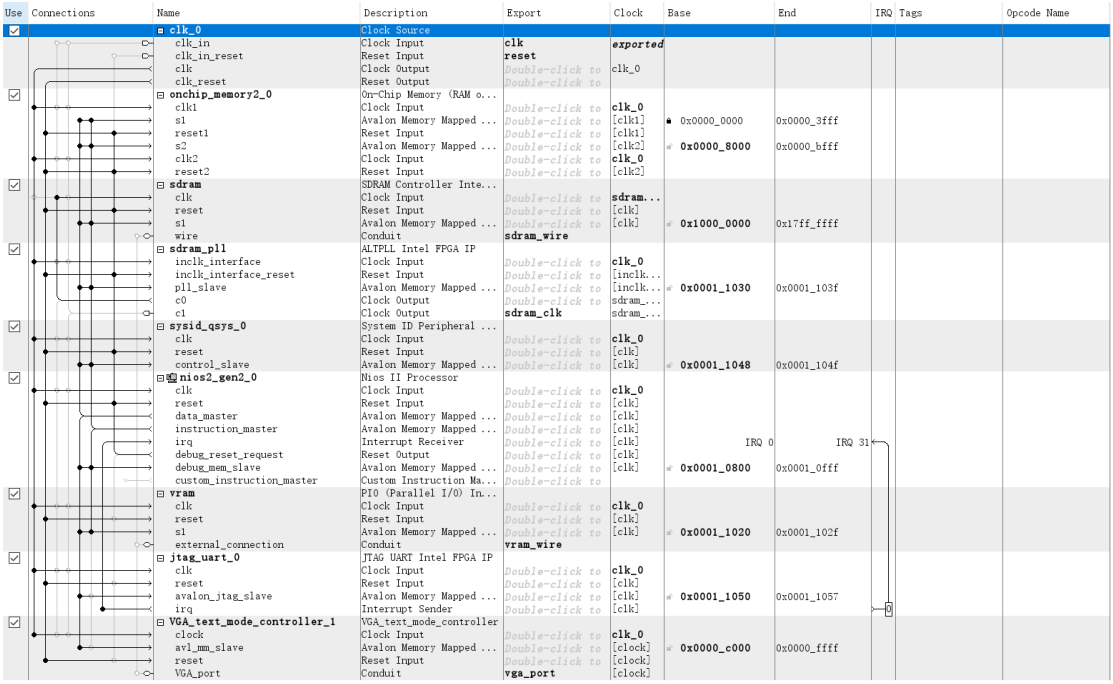
Module: vga_text_avl_interface.sv
Inputs: CLK, RESET, AVL_READ, AVL_WRITE, AVL_CS, [3:0] AVL_BYTE_EN, [11:0] AVL_ADDR, [31:0] AVL_WRITEDATA,
Outputs: [31:0] AVL_READDATA, [3:0] red, [3:0] green, [3:0] blue, hs, vs, sync, blank, pixel_clk
Description: The vga_text_avl_interface module connects to the Avalon Memory-Mapped bus to handle a text-based VGA display on the DE2-115 board. It takes care of both the video memory (VRAM) and a control register used for managing display settings. The VRAM is set up in row-by-row order and supports an 80x30 character layout. This module handles read and write operations to VRAM and the control register depending on the Avalon-MM signals. It figures out where data should go by checking the address, then sends the right info to either VRAM or the control logic. It also takes care of producing the signals needed to show the VRAM contents on the screen.
Purpose: This module handles both read and write actions using the Avalon-MM bus, updating either the video memory or the control register depending on what's being accessed. It creates the VGA output by reading character and control info from VRAM, turning that data into visible text on the screen. The display's look—like text and background colors—can be changed directly in hardware by writing new values to the control register.

Module: font_rom.sv
Inputs: [10:0] addr
Outputs: [7:0] data
Description: The font_rom module acts as a ROM that stores the pixel patterns for different characters used on the screen. It contains preloaded images of characters, usually based on part of the IBM codepage 437 set. Each character is drawn using an 8x16 pixel grid, where each line is stored as an 8-bit number—each bit representing whether a pixel in that row is lit up or not.
Purpose: The font is stored as a customizable array filled with hardcoded binary values that outline how each character looks in black and white. Every entry matches one row in a character's design, allowing for detailed character shapes. The module takes an 11-bit address to look up 8-bit values, each representing one line of a character stored in the ROM. The output gives back the pixel data for that specific row, which can then be used by the display system or other modules handling graphics.



Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Tags	Opcode Name
☑		clk_0	Clock Source							
		clk_in	Clock Input	clk	exported					
		clk_in_reset	Reset Input	reset						
		clk	Clock Output	Double-click to	clk_0					
		clk_reset	Reset Output	Double-click to						
☑		onchip_memo...	On-Chip Memory (RAM o...							
		clk1	Clock Input	Double-click to	clk_0					
		s1	Avalon Memory Mapped ...	Double-click to	[clk1]	0x0000_0000	0x0000_000f			
		reset1	Reset Input	Double-click to	[clk1]					
☑		sdram	SDRAM Controller Inte...							
		clk	Clock Input	Double-click to	sdram...					
		reset	Reset Input	Double-click to	[clk]					
		s1	Avalon Memory Mapped ...	Double-click to	[clk]	0x1000_0000	0x17ff_ffff			
		wire	Conduit	sdram_wire						
☑		sdram_pll	ALTPLL Intel FPGA IP							
		inclk_inter...	Clock Input	Double-click to	clk_0					
		inclk_inter...	Reset Input	Double-click to	[inclk...					
		pll_slave	Avalon Memory Mapped ...	Double-click to	[inclk...	0x0000_0030	0x0000_003f			
		c0	Clock Output	Double-click to	sdram_...					
		c1	Clock Output	sdram_clk	sdram_...					
☑		sysid_qsys_0	System ID Peripheral ...							
		clk	Clock Input	Double-click to	clk_0					
		reset	Reset Input	Double-click to	[clk]					
		control_slave	Avalon Memory Mapped ...	Double-click to	[clk]	0x0000_0048	0x0000_004f			
☑		nios2_gen2_0	Nios II Processor							
		clk	Clock Input	Double-click to	clk_0					
		reset	Reset Input	Double-click to	[clk]					
		data_master	Avalon Memory Mapped ...	Double-click to	[clk]					
		instruction...	Avalon Memory Mapped ...	Double-click to	[clk]					
		irq	Interrupt Receiver	Double-click to	[clk]			IRQ 0	IRQ 31	
		debug_reset...	Reset Output	Double-click to	[clk]					
		debug_mem_s...	Avalon Memory Mapped ...	Double-click to	[clk]	0x0000_2000	0x0000_27ff			
		custom_inst...	Custom Instruction Ma...	Double-click to						
☑		keycode	PIO (Parallel I/O) In...							
		clk	Clock Input	Double-click to	clk_0					
		reset	Reset Input	Double-click to	[clk]					
		s1	Avalon Memory Mapped ...	Double-click to	[clk]	0x0000_0020	0x0000_002f			
		external_co...	Conduit	keycode						
☑		jtag_uart_0	JTAG UART Intel FPGA IP							
		clk	Clock Input	Double-click to	clk_0					
		reset	Reset Input	Double-click to	[clk]					
		avalon_jtag...	Avalon Memory Mapped ...	Double-click to	[clk]	0x0000_0050	0x0000_0057			
		irq	Interrupt Sender	Double-click to	[clk]			0		
☑		VGA_text_mo...	VGA_text_mode_controller							
		clock	Clock Input	Double-click to	clk_0					
		avl_mm_slave	Avalon Memory Mapped ...	Double-click to	[clock]	0x0000_1000	0x0000_1fff			
		reset	Reset Input	Double-click to	[clock]					
		VGA_port	Conduit	vga_port	[clock]					

Figure 3: Platform Designer for 9.1



Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Tags	Opcode Name
☑		clk_0	Clock Source							
		clk_in	Clock Input	clk	exported					
		clk_in_reset	Reset Input	reset						
		clk	Clock Output	Double-click to	clk_0					
		clk_reset	Reset Output	Double-click to						
☑		onchip_memory2_0	On-Chip Memory (RAM o...							
		clk1	Clock Input	Double-click to	clk_0					
		s1	Avalon Memory Mapped ...	Double-click to	[clk1]	0x0000_0000	0x0000_3fff			
		reset1	Reset Input	Double-click to	[clk1]					
		s2	Avalon Memory Mapped ...	Double-click to	[clk2]	0x0000_8000	0x0000_bfff			
		clk2	Clock Input	Double-click to	clk_0					
		reset2	Reset Input	Double-click to	[clk2]					
☑		sdram	SDRAM Controller Inte...							
		clk	Clock Input	Double-click to	sdram...					
		reset	Reset Input	Double-click to	[clk]					
		s1	Avalon Memory Mapped ...	Double-click to	[clk]	0x1000_0000	0x17ff_ffff			
		wire	Conduit	sdram_wire						
☑		sdram_pll	ALTPLL Intel FPGA IP							
		inclk_interface	Clock Input	Double-click to	clk_0					
		inclk_interface_reset	Reset Input	Double-click to	[inclk...					
		pll_slave	Avalon Memory Mapped ...	Double-click to	[inclk...	0x0001_1030	0x0001_103f			
		c0	Clock Output	Double-click to	sdram_...					
		c1	Clock Output	sdram_clk	sdram_...					
☑		sysid_qsys_0	System ID Peripheral ...							
		clk	Clock Input	Double-click to	clk_0					
		reset	Reset Input	Double-click to	[clk]					
		control_slave	Avalon Memory Mapped ...	Double-click to	[clk]	0x0001_1048	0x0001_104f			
☑		nios2_gen2_0	Nios II Processor							
		clk	Clock Input	Double-click to	clk_0					
		reset	Reset Input	Double-click to	[clk]					
		data_master	Avalon Memory Mapped ...	Double-click to	[clk]					
		instruction_master	Avalon Memory Mapped ...	Double-click to	[clk]					
		irq	Interrupt Receiver	Double-click to	[clk]			IRQ 0	IRQ 31	
		debug_reset_request	Reset Output	Double-click to	[clk]					
		debug_mem_slave	Avalon Memory Mapped ...	Double-click to	[clk]	0x0001_0800	0x0001_0fff			
		custom_instruction_master	Custom Instruction Ma...	Double-click to						
☑		vram	PIO (Parallel I/O) In...							
		clk	Clock Input	Double-click to	clk_0					
		reset	Reset Input	Double-click to	[clk]					
		s1	Avalon Memory Mapped ...	Double-click to	[clk]	0x0001_1020	0x0001_102f			
		external_connection	Conduit	vram_wire						
☑		jtag_uart_0	JTAG UART Intel FPGA IP							
		clk	Clock Input	Double-click to	clk_0					
		reset	Reset Input	Double-click to	[clk]					
		avalon_jtag_slave	Avalon Memory Mapped ...	Double-click to	[clk]	0x0001_1050	0x0001_1057			
		irq	Interrupt Sender	Double-click to	[clk]				0	
☑		VGA_text_mode_controller_1	VGA_text_mode_controller							
		clock	Clock Input	Double-click to	clk_0					
		avl_mm_slave	Avalon Memory Mapped ...	Double-click to	[clock]	0x0000_c000	0x0000_ffff			
		reset	Reset Input	Double-click to	[clock]					
		VGA_port	Conduit	vga_port	[clock]					

Figure 4: Platform Designer for 9.2

5. Design Resources and Statistics

Week 1

LUT	31223
DSP	0
Memory (BRAM)	11220 bits
Flip-Flop	21754
Frequency	52.03MHz
Static Power	97.42mW
Dynamic Power	300.66mW
Total Power	455.01mW

Table 1: Design Resources and Statistics week 1

Week 2

LUT	3621
DSP	0
Memory (BRAM)	76777 bits
Flip-Flop	2190
Frequency	80.02MHz
Static Power	96.32mW
Dynamic Power	57.90mW
Total Power	211.19mW

Table 2: Design Resources and Statistics week 2

Week 2's design significantly reduced Logic Elements and Flip-Flops by replacing 601 registers from Week 1 with dual-port memory, increasing efficiency and cutting power consumption nearly in half. Although it limits data manipulation flexibility, it improves operating speed and compile time, making it better suited for high-performance, low-power applications.

6. Conclusion

Functionality Discussion

Week 1 proceeded smoothly, but Lab 9.2 introduced several challenges. Initially, we attempted to implement on-chip memory using a SystemVerilog file, which caused routing errors as the compiler failed to recognize it. We resolved this by configuring the memory through the Altera IP GUI. Another issue arose when writing to the palette registers—both stored colors were being altered unintentionally. We fixed this by redesigning the palette to store one color per register.

Potential Extensions

A possible extension is using the font_rom to store sequences of glyphs or images, enabling basic animations. While the current system is primarily used for the final project's menu interface, it could be expanded for more interactive or visual features.

Lab Difficulty and Improvements

We found the lab to be appropriately challenging for an undergraduate course. Although it took time to fully understand the requirements, the provided instructions were clear and thorough, helping us complete the design effectively.